

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

Đề tài: Xây dựng game Tetris sử dụng KIT STM32F429I-DISC1

Lớp : 168505
Học phần : Hệ nhúng
Mã học phần : IT4210
Giảng viên hướng dẫn : TS. Ngô Lam Trung

Danh sách thành viên nhóm 34:

Họ và tên	Mã số sinh viên	Email
Nguyễn Tuấn Cảnh	20225168	canh.nt225168@sis.hust.edu.vn
Đỗ Văn Việt	20225428	viet.dv225428@sis.hust.edu.vn

Hà Nội, tháng 7 năm 2026

MỤC LỤC

LỜI NÓI ĐẦU	4
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	6
1.1. Lý do chọn đề tài.....	6
1.2. Mục tiêu.....	7
1.3. Ý tưởng thực hiện.....	8
1.4. Phương pháp nghiên cứu.....	8
CHƯƠNG 2: HỆ THỐNG PHẦN CỨNG.....	9
2.1. Kiến trúc hệ thống.....	9
2.2. Tổng quan về Kit STM32F429I-DISC1	9
2.2.1. Phần cứng của Kit.....	10
2.2.2. Tính năng, giao thức kết nối của Kit	10
2.2.3. Vị trí các thành phần trên Kit và kích thước	11
2.3. Ngoại vi mở rộng: nút bấm, joystick, RNG và buzzer.....	13
2.3.1. Danh sách linh kiện (BOM)	14
2.3.2. Bảng gán chân (pin-map) và sơ đồ đấu nối	15
2.3.3. Cấu hình RNG phần cứng sinh khối ngẫu nhiên (Yêu cầu kỹ thuật 1).....	18
2.3.4. Cấu hình PWM phát âm thanh qua buzzer (Yêu cầu kỹ thuật 2).....	18
CHƯƠNG 3: XÂY DỰNG CHƯƠNG TRÌNH VÀ TRIỂN KHAI CÀI ĐẶT.....	19
3.1. Sơ đồ hệ thống.....	19
3.2. Luồng hệ thống.....	20
3.2.1. Nhận dữ liệu từ tác nhân người chơi	20
3.2.2. Xử lý logic của game và xây dựng giao diện	23

3.3. Luật chơi và thuật toán.....	24
3.3.1. Bàn chơi và khối.....	24
3.3.2. Va chạm, khoá khối và xoá hàng.....	24
3.3.3. Tính điểm, cấp độ và trọng lực.....	24
3.3.4. Hold và Ghost piece	25
3.4. Chế độ chơi	25
3.5. Lưu điểm cao (High score).....	26
CHƯƠNG 4: ĐÁNH GIÁ TỔNG QUAN.....	28
4.1. 4.1. Sản phẩm demo	28
4.2. Đánh giá hệ thống	32
4.3. Hướng phát triển	32
PHỤ LỤC.....	34
1. Phân công nhiệm vụ.....	34
2. Tài liệu tham khảo	35
3. Mã nguồn (source code) và các resource của đồ án.....	35

LỜI NÓI ĐẦU

Công nghệ thông tin ngày càng phát triển và có vai trò hết sức quan trọng không thể thiếu trong cuộc sống hiện đại, nó đã thay đổi mọi khía cạnh của cuộc sống, từ việc giao tiếp, giải trí, đến học tập và công việc. Chúng ta sống trong một thế giới với hàng tỷ thiết bị kết nối với Internet, tạo nên mạng lưới thông tin phức tạp. Nhưng không chỉ có những thiết bị lớn như máy tính cá nhân và điện thoại thông minh, công nghệ thông tin còn hiện diện ẩn mình trong những hệ thống nhỏ gọn và tích hợp, được gọi là hệ thống nhúng.

Hệ thống nhúng là những thiết bị và hệ thống tích hợp các vi xử lý, cảm biến, và giao tiếp mạng. Điển hình cho các ứng dụng của hệ thống nhúng là các thiết bị di động, các thiết bị y tế thông minh, đồ gia dụng kết nối Internet (Internet of Things - IoT) và nhiều ứng dụng khác.

Sự phát triển của công nghệ đã thúc đẩy sự tiến bộ của hệ nhúng. Các bộ vi xử lý ngày càng mạnh mẽ và nhỏ gọn, công nghệ bộ nhớ và lưu trữ phát triển nhanh chóng, đồng thời với đó là xu hướng phần mềm mã nguồn mở và các công cụ phát triển hệ thống nhúng. Tất cả những tiến bộ này đã tạo điều kiện thuận lợi cho việc phát triển các ứng dụng hệ thống nhúng phức tạp và đa dạng.

Với sự hấp dẫn của lĩnh vực và những thách thức còn đang ở phía trước, với niềm đam mê, mong muốn được học hỏi các công nghệ, tiếp xúc với các bài toán của Hệ nhúng, nhóm chúng em đã quyết định lựa chọn đề tài “Xây dựng game Tetris sử dụng KIT STM32F429I-DISC1” cho Đồ án môn học của mình.

Đồ án môn học của nhóm chúng em bao gồm 4 nội dung chính:

1. Tổng quan đề tài
2. Hệ thống phần cứng
3. Xây dựng chương trình và triển khai cài đặt
4. Đánh giá tổng quan

Mặc dù đã cố gắng hoàn thiện sản phẩm nhưng không thể tránh khỏi những thiếu hụt về kiến thức và sai sót trong kiểm thử. Chúng em rất mong nhận được những nhận xét thẳng thắn, chi tiết đến từ thầy để tiếp tục hoàn thiện hơn nữa. Cuối cùng, nhóm

chúng em xin được gửi lời cảm ơn đến thầy TS. Ngô Lam Trung đã hướng dẫn chúng em trong suốt quá trình hoàn thiện Đồ án môn học. Nhóm chúng em xin chân thành cảm ơn thầy.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Các thiết bị điều khiển và các hệ thống nhúng ngày một nhiều và trở nên phổ biến quanh cuộc sống thường ngày với nền văn minh hiện đại. Thực tế mỗi khía cạnh của hoạt động hằng ngày đều bị chi phối bởi một vài loại hệ thống điều khiển. Dễ dàng tìm thấy các thiết bị điều khiển cho các máy móc, công cụ, điều khiển máy tính, các hệ thống giao thông, hệ thống năng lượng robot,... rồi các ứng dụng vĩ mô hơn như công nghệ kỹ thuật liên quan đến vũ khí công nghệ cao, ngành hàng không vũ trụ, hay đơn giản là các ví dụ về các phương tiện đi lại, đồ gia dụng, dân dụng mỗi người sử dụng hằng ngày.

Việc xây dựng hệ thống nhúng là một thách thức thú vị và có ý nghĩa rất lớn trong việc nâng cao kỹ năng và hiểu sâu hơn về cách hoạt động của các thiết bị điện tử thông minh. Trong Đồ án học phần này, nhóm chúng em quyết định lựa chọn đề tài “Xây dựng game Tetris sử dụng KIT STM32F429I-DISC1” với lý do sau:

- Tetris: trò chơi xếp hình kinh điển, luật chơi tuy đơn giản nhưng đòi hỏi xử lý thời gian thực chặt chẽ (trọng lực khối, va chạm, xoá hàng, tăng tốc theo level) — một bài toán vừa quen thuộc để đánh giá đúng/sai khi kiểm thử, vừa đủ thử thách để khai thác các ngoại vi nâng cao của KIT.
- Ứng dụng thực tiễn: KIT STM32F429I-DISC1 là 1 board phổ biến và mạnh mẽ được sử dụng rộng rãi trong việc phát triển các hệ thống nhúng. Xây dựng trò chơi Tetris trên nền tảng này, kết hợp thư viện đồ hoạ TouchGFX và hệ điều hành thời gian thực FreeRTOS, giúp nhóm hiểu rõ hơn về cách làm việc với kit phần cứng và khai thác các ngoại vi như RNG, ADC, PWM, Flash nội.
- Học hỏi và phát triển kỹ năng: Đề tài đòi hỏi nhóm phải áp dụng kiến thức về vi điều khiển, cấu trúc dữ liệu, giao tiếp ngoại vi (RNG, ADC, PWM, GPIO, Flash) và lập trình đa nhiệm (FreeRTOS). Qua việc học tập trên lớp Lý thuyết và làm việc tại lớp Thực hành, chúng em có cơ hội rèn luyện và nâng cao kỹ năng lập trình phần cứng cũng như tư duy thiết kế hệ thống thời gian thực.

- Xây dựng sản phẩm cuối cùng: Mục tiêu cuối cùng của đề tài là tạo ra một trò chơi Tetris hoàn chỉnh trên hệ thống nhúng, hoạt động ổn định. Điều này giúp chúng em hiểu quy trình phát triển phần mềm nhúng và chuẩn bị cho các dự án thực tế trong tương lai.

Với những lý do trên, trong khuôn khổ Đồ án môn học, nhóm em với đề tài trên rất mong muốn triển khai một hệ thống nhúng sẽ mang lại những trải nghiệm học tập và thực hành đáng giá, đồng thời góp phần thúc đẩy sự phát triển và ứng dụng của công nghệ nhúng trong cuộc sống hàng ngày.

1.2. Mục tiêu

Trong quá trình học tập học phần Hệ nhúng, nhận thấy bối cảnh và những nhu cầu kể trên, nhóm chúng em đặt ra mục tiêu phải xây dựng được sản phẩm với các tiêu chí:

1. Ứng dụng được kiến thức về xây dựng các hệ thống nhúng đơn giản.
2. Phải là một sản phẩm được xem là “thông minh”.
3. Xây dựng giao diện người dùng (User Interface – UI): Phát triển giao diện TouchGFX trực quan gồm Menu chọn chế độ chơi, màn hình chơi (HUD hiển thị Score/Level/Lines/Next/Hold), Pause và Game Over, giúp người chơi tương tác với Tetris một cách dễ dàng.
4. Xử lý luật chơi: Xác định và cài đặt luật chơi Tetris — 7 khối tetromino, xoay, va chạm, rơi theo trọng lực, xoá hàng, tính điểm, tăng level, hold và ghost piece.
5. Điều khiển ngoại vi: Sử dụng KIT STM32F429I-DISC1 để điều khiển các ngoại vi cần thiết cho trò chơi: màn hình LCD hiển thị, 9 nút bấm rời và joystick analog để nhận lệnh điều khiển, RNG phần cứng để sinh khối ngẫu nhiên, buzzer PWM để phát âm thanh. Thông qua việc tương tác với các ngoại vi này, người chơi có thể thấy và thao tác với trò chơi một cách trực quan và linh hoạt.

6. Kiểm thử và sửa lỗi: Tiến hành kiểm thử chéo để đảm bảo tính đúng đắn và ổn định của trò chơi. Nếu phát hiện lỗi, tiến hành sửa chúng và kiểm tra lại để đảm bảo hoạt động chính xác.

1.3. Ý tưởng thực hiện

Trò chơi Tetris là trò chơi xếp hình (puzzle) dạng khối rơi (falling-block): người chơi điều khiển các khối tetromino rơi từ trên xuống, di chuyển và xoay để lấp đầy các hàng ngang trên bàn chơi 10×20 ô; hàng đầy sẽ được xóa và tính điểm, tốc độ rơi tăng dần theo level khiến trò chơi ngày càng khó.

Trò chơi được xây dựng trên KIT STM32F429I-DISC1, kết hợp thư viện đồ họa TouchGFX và hệ điều hành thời gian thực FreeRTOS. Bộ STM32F429I-DISCOVERY tận dụng khả năng của vi điều khiển hiệu suất cao STM32F429 (lõi ARM Cortex-M4), cho phép phát triển các ứng dụng phong phú với giao diện đồ họa tiên tiến; FreeRTOS đảm bảo việc đọc ngoại vi (nút bấm, joystick), xử lý luật chơi, vẽ giao diện và phát âm thanh diễn ra song song mà không làm chậm nhau.

1.4. Phương pháp nghiên cứu

- Đi từ những kiến thức cơ bản về board mạch, cách vận hành cấu hình đến lập trình và nhúng.
- Sử dụng STM32CubeIDE để code, debug và biên dịch.
- Chuẩn bị kit STM32F429I-DISC1 và các linh kiện cần thiết (joystick analog KY-023, nút bấm rời, buzzer thụ động KY-006, breadboard, dây jumper).
- Tìm hiểu về Kit STM32F429I-DISC1, các linh kiện, thiết bị ngoại vi thông qua các datasheet của Kit và các tài liệu liên quan trên Internet và Youtube.

CHƯƠNG 2: HỆ THỐNG PHẦN CỨNG

2.1. Kiến trúc hệ thống

Hệ thống gồm 4 thành phần chính:

1. Kit STM32F429I-DISC1
2. Mini USB-cable
3. Khối ngoại vi mở rộng (joystick, nút bấm, buzzer trên breadboard)
4. Dây nối

2.2. Tổng quan về Kit STM32F429I-DISC1



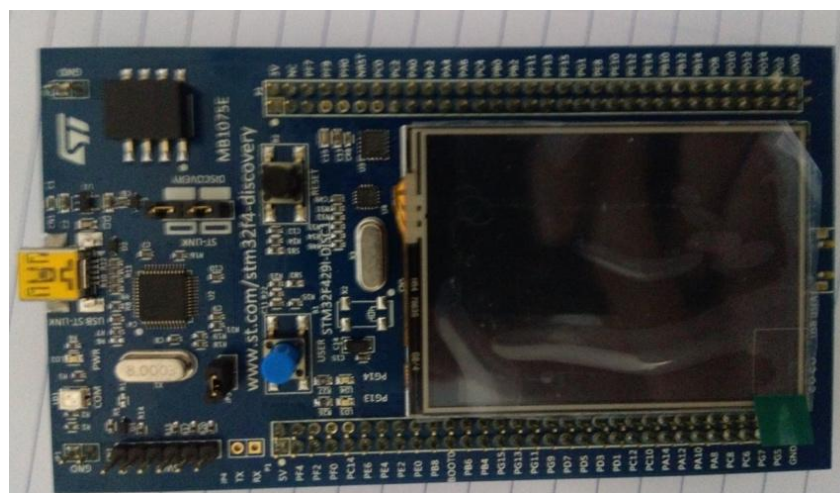
Hình 1: Mặt trước và mặt sau của Kit

Bộ STM32F429I-DISC1 cho phép người dùng dễ dàng phát triển các ứng dụng với MCU hiệu suất cao STM32F429 với lõi ARM®Cortex®-M4.

Kit bao gồm một công cụ gỡ lỗi nhúng ST-LINK / V2-B, màn hình LCD TFT QVGA 2,4 ", hỗ trợ SDRAM 64 Mbit, con quay hồi chuyển ST MEMS hỗ trợ trong việc nhận diện góc quay, đầu nối micro-AB USB OTG, đèn LED và nút ấn.

2.2.1. Phần cứng của Kit

- Kit có tích hợp Micro Vi điều khiển (Micro Controller Unit – MCU) là đơn vị xử lý nhỏ, nó được tích hợp toàn bộ các bộ nhớ như ROM, RAM, các Port truy xuất, giao tiếp ngoại vi trực tiếp trên một con chip hết sức nhỏ gọn.
- Màn hình QVGA TFT LCD 2.4 hiển thị màu.
- 6 Led bao gồm:
 - LD1 (đỏ/xanh) cho giao tiếp USB
 - LD2 (đỏ) dành cho cấp nguồn 3.3V
 - 2 Led người dùng: LD3 (xanh) và LD4 (đỏ)
 - 2 Led USB OTG: LD5 (xanh) VBUS và LD6 (đỏ) OC
- 2 nút bấm: User và Reset
- Cổng kết nối USB OTG với chuẩn micro-AB
- Cảm biến con quay hồi quy xoay 3 trục xyz
- Cổng USB cấp nguồn 3V hoặc 5V



Hình 2: KIT STM32F429I-DISCOVERY

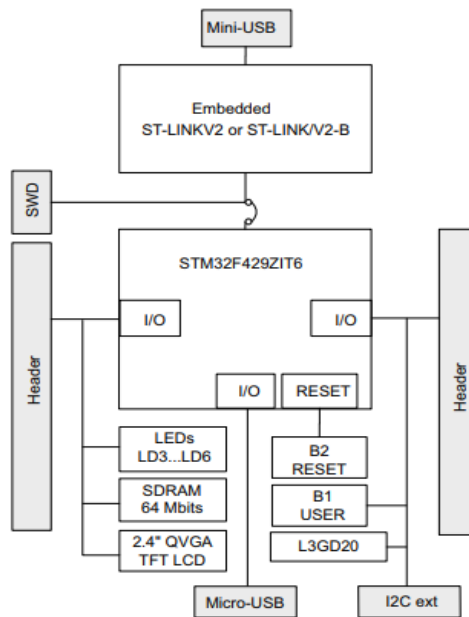
2.2.2. Tính năng, giao thức kết nối của Kit

- Tính năng tích hợp ST-Link/V2 hỗ trợ 2 chế độ nạp code chạy và debug

- Tính năng mbed-enabled : nhằm đảm bảo các nhà phát triển có thể sử dụng các công nghệ Mbed (Mbed OS, Mbed Linux và các công cụ khác) trên một loạt các phần cứng đủ điều kiện.
- Tính năng cảm biến con quay hồi quy xoay Kit theo 3 trục xyz
- Tích hợp màn hình Led có hỗ trợ cảm ứng
- Kết nối USB có các chức năng chính:
 - Cổng debug
 - Cổng COM ảo
 - Lưu trữ Mass
 - Nạp nguồn cho Kit
 - Giao tiếp USB OTG chuẩn micro-AB

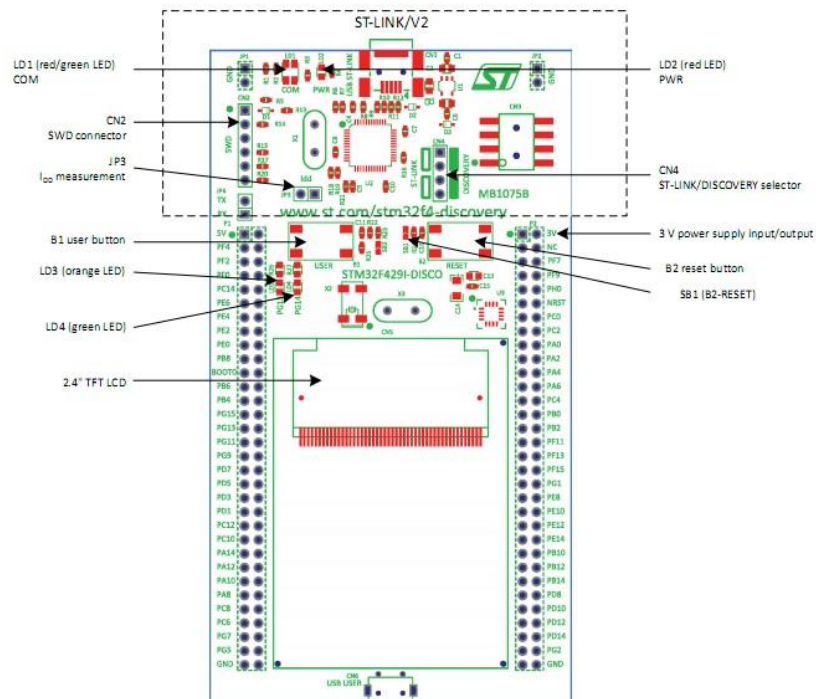
2.2.3. Vị trí các thành phần trên Kit và kích thước

2.2.3.1. Sơ đồ khối của Kit

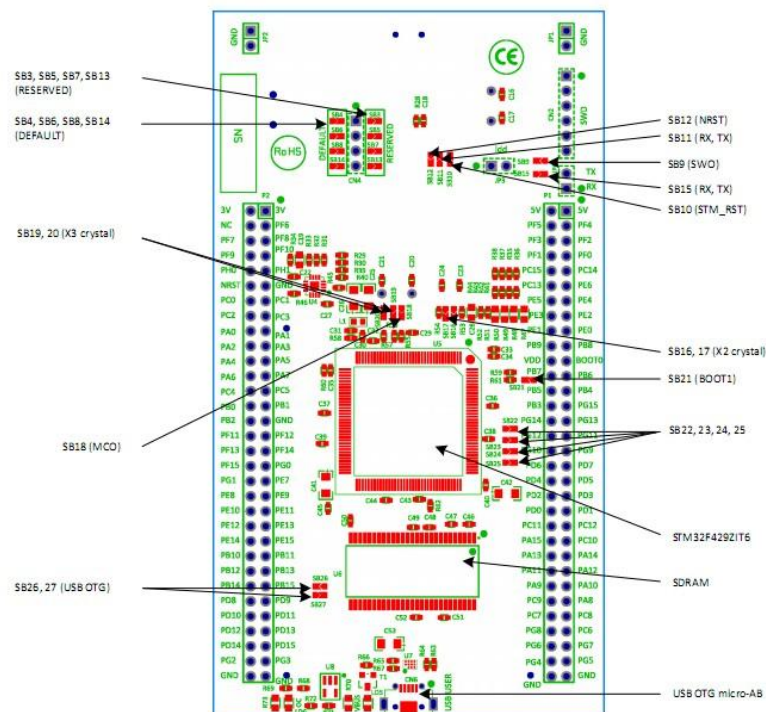


Hình 3: Sơ đồ khối các thành phần của Kit

2.2.3.2. Vị trí của các thành phần trên mạch

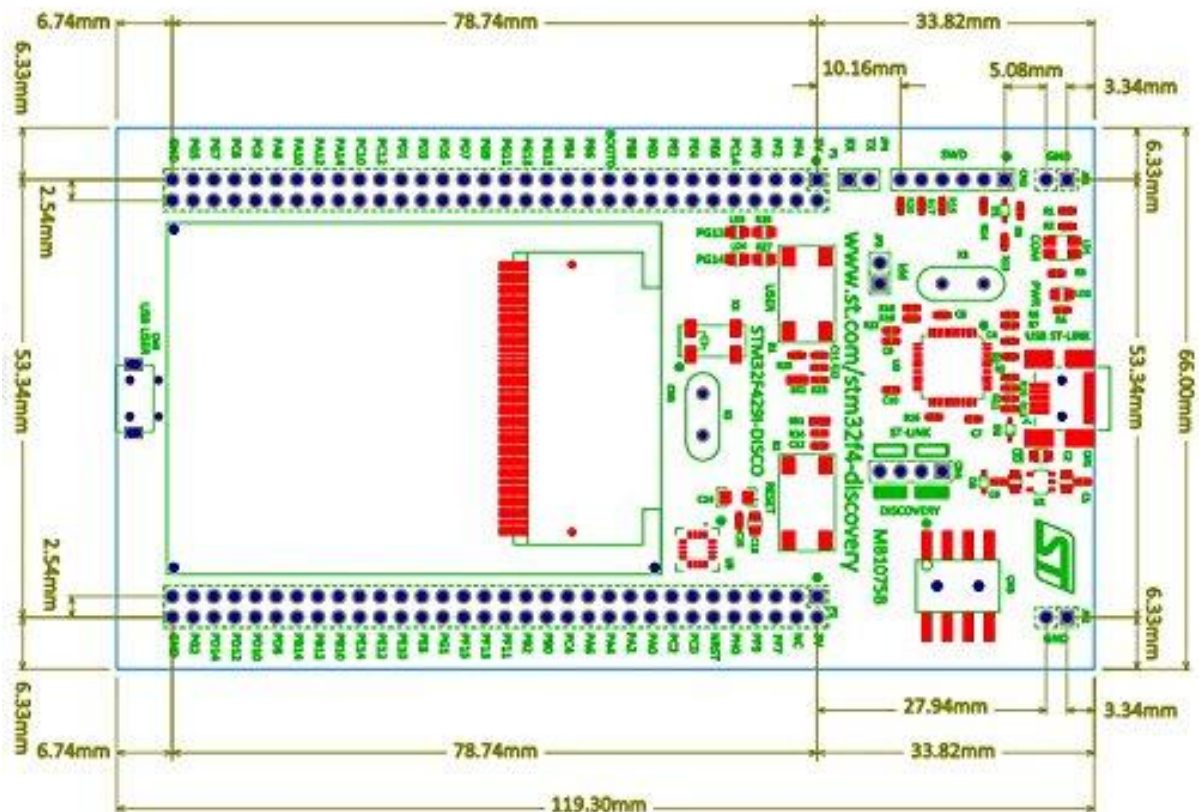


Hình 4: Vị trí các thành phần của Kit từ phía trên



Hình 5: Vị trí các thành phần của Kit từ phía dưới

2.2.3.3. Kích thước của Kit



Hình 6: Kích thước của Kit (đơn vị: mm)

2.3. Ngoại vi mở rộng: nút bấm, joystick, RNG và buzzer

Để đáp ứng 2 yêu cầu kỹ thuật bắt buộc của đề tài - sinh khởi ngẫu nhiên bằng RNG phần cứng và phát âm thanh - nhóm mở rộng KIT với 1 joystick analog, 9 nút bấm rời và 1 buzzer thụ động, đấu nối qua breadboard vào 2 header mở rộng P1/P2 của board.

Cách đếm nút dùng trong báo cáo: “9 nút bấm rời” là tổng số nút vật lý cắm thêm trên breadboard, gồm 7 nút cho người chơi 1 (LEFT, RIGHT, ROT_CW, ROT_CCW, SOFT_DROP, HARD_DROP, HOLD) và 2 nút riêng cho người chơi 2 ở chế độ 2P Versus (HOLD_P2 = PB7, ROT_CCW_P2 = PC12). Bên cạnh đó, joystick có sẵn 1 nút nhấn cần (SW) và board có sẵn nút USER (PA0) được dùng làm nút PAUSE, nên không tính vào số nút rời. Ở các mục sau, cụm “8 nút bấm chính” chỉ nhóm 8 nút GPIO active-LOW mà InputTask đọc chung mỗi vòng (7 nút của người chơi 1 và nút SW của joystick); 2 nút PvP và nút PAUSE được xử lý riêng.

2.3.1. Danh sách linh kiện (BOM)

Bảng dưới liệt kê các linh kiện cần chuẩn bị thêm ngoài KIT:

STT	Linh kiện	Số lượng & ghi chú
1	Joystick analog KY-023 (2 trục VRx/VRy + nút nhấn SW)	1 — cấp nguồn 3V3, KHÔNG dùng 5V
2	Nút nhấn rời (tact switch 4 chân)	9 — 7 nút điều khiển chính (Trái, Phải, Xoay CW, Xoay CCW, Soft drop, Hard drop, Hold P1) + 2 nút dành cho người chơi 2 ở chế độ 2P Versus (Hold P2, Xoay CCW P2). Pause dùng nút USER (PA0) có sẵn trên board.
3	Buzzer thụ động (passive, kiểu KY-006)	1 — bắt buộc loại thụ động để điều chỉnh bằng PWM, KHÔNG dùng active buzzer
4	Breadboard	1 — gắn nút, joystick, buzzer
5	Dây jumper đực–cái (M–F)	~20 — nối header P1/P2 của KIT ra breadboard
6	Dây jumper đực–đực (M–M)	~10 — nối GND chung, đi dây trên breadboard
7	Điện trở ~100 Ω (tùy chọn)	1 — nối tiếp buzzer cho êm/giảm dòng

8	Cáp mini-USB	1 — nạp firmware + cấp nguồn cho KIT
---	--------------	--------------------------------------

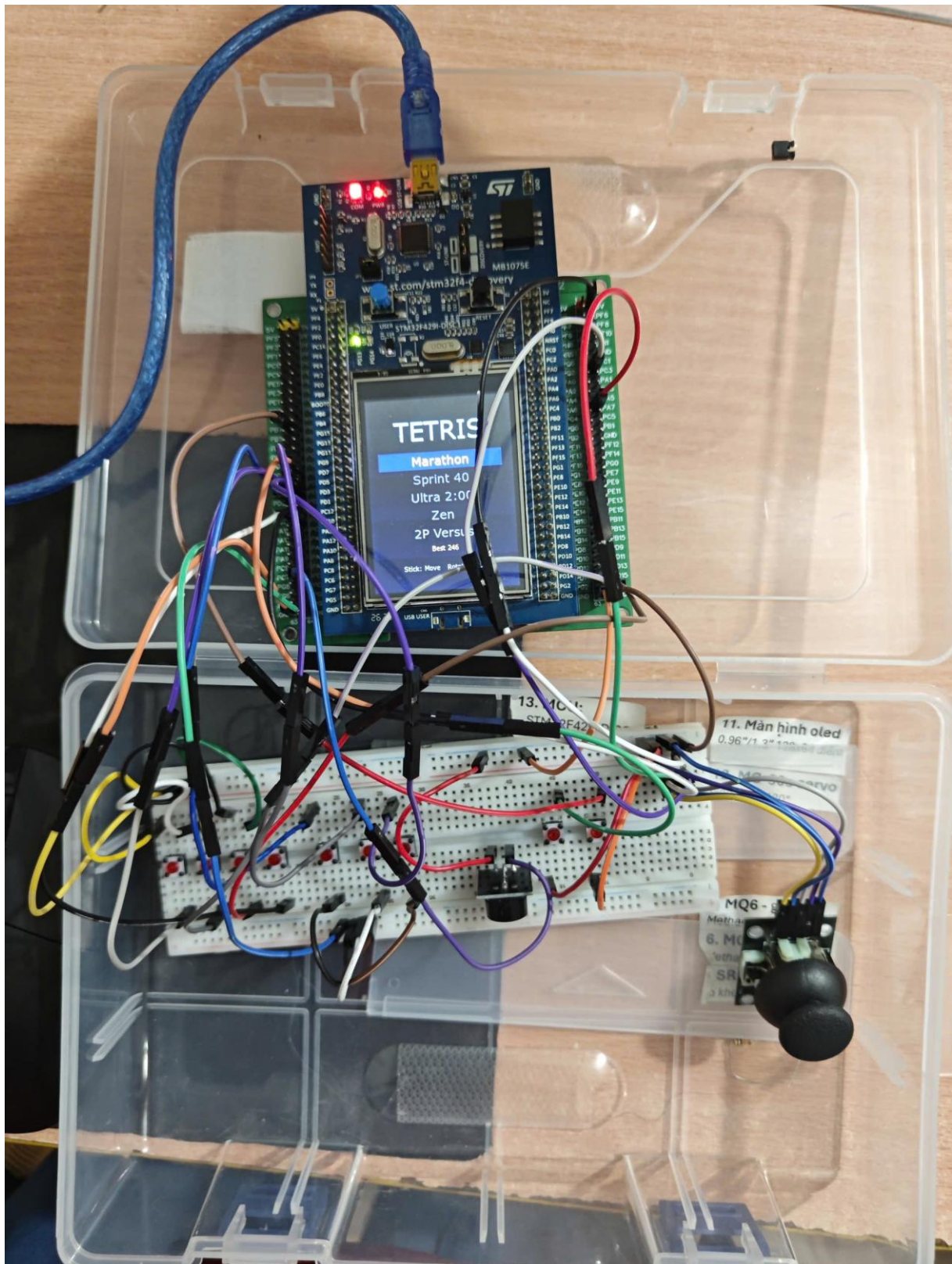
2.3.2. Bảng gán chân (pin-map) và sơ đồ đấu nối

Bảng gán chân dưới đây đã được kiểm chứng chéo với sơ đồ nguyên lý board (UM1670) để tránh các chân bị nối cứng với LCD/SDRAM/con quay hồi chuyển trên board:

#	Tín hiệu	Chân	Chức năng MCU	Ghi chú
1	Joystick VRx (trục X)	PC3	ADC1_IN13	Header P2
2	Joystick VRy (trục Y)	PA7	ADC1_IN7	Header P2
3	Buzzer (KY-006)	PC8	TIM3_CH3 (PWM)	Header P1
4	Nút LEFT	PG2	GPIO Input, Pull-up	active-LOW
5	Nút RIGHT	PG3	GPIO Input, Pull-up	active-LOW
6	Nút ROTATE CW	PC11	GPIO Input, Pull-up	active-LOW (đổi từ PD6 vì đụng LCD)
7	Nút ROTATE CCW	PD4	GPIO Input, Pull-up	active-LOW
8	Nút SOFT DROP	PD2	GPIO Input, Pull-up	active-LOW
9	Nút HARD DROP	PD7	GPIO Input, Pull-up	active-LOW

10	Nút HOLD (P1)	PD5	GPIO Input, Pull-up	active-LOW
11	Joystick SW (nhấn cần = Xoay CW)	PG9	GPIO Input, Pull-up	active-LOW
12	PAUSE	PA0	GPIO Input, không pull	active-HIGH (nút USER có sẵn)
13	Nút HOLD (P2, PvP)	PB7	GPIO Input, Pull-up	active-LOW
14	Nút ROTATE CCW (P2, PvP)	PC12	GPIO Input, Pull-up	active-LOW
15	LED xanh (có tiếng)	PG13	GPIO Output	LD3 có sẵn trên board
16	LED đỏ (tắt tiếng)	PG14	GPIO Output	LD4 có sẵn trên board

Lưu ý khi đấu nối: 8 nút bấm chính (mục 4–11) đều thuộc loại active-LOW (nhấn = mức 0), dùng pull-up nội của STM32 nên không cần điện trở ngoài; joystick cấp nguồn 3V3 (không dùng 5V). Riêng nút PAUSE (PA0, nút USER có sẵn) active-HIGH (nhấn = mức 1) và không dùng pull nội. Các chân sau tuyệt đối không được dùng làm nút/joystick vì đã nối cứng với LCD/SDRAM/con quay hồi chuyển trên board: PD6, PC10, PA1, PA2, PC4, PC5, PC13, PD11.



Hình 7: Mạch đấu nối joystick, nút bấm và buzzer thực tế

2.3.3. Cấu hình RNG phần cứng sinh khối ngẫu nhiên (Yêu cầu kỹ thuật 1)

Bộ sinh số ngẫu nhiên phần cứng (RNG) của STM32F429 được bật trong STM32CubeMX (Security/Analog → RNG → Activated); ngoại vi này không cần chân ra ngoài và dùng chung miền clock 48 MHz sẵn có của TouchGFX (PLLQ), không cần chỉnh lại cây clock của board (RNG chỉ cần $\text{clock} > \text{HCLK}/16$). RNG được dùng để sinh khối tetromino theo thuật toán 7-bag (xáo trộn Fisher-Yates một túi gồm đủ 7 loại khối), đảm bảo khối ra ngẫu nhiên nhưng công bằng — không lặp lại quá gần hay “đổi” một loại khối nào, đúng yêu cầu kỹ thuật bắt buộc số 1 của đề tài (không dùng hàm rand()/LCG phần mềm trên board thật).

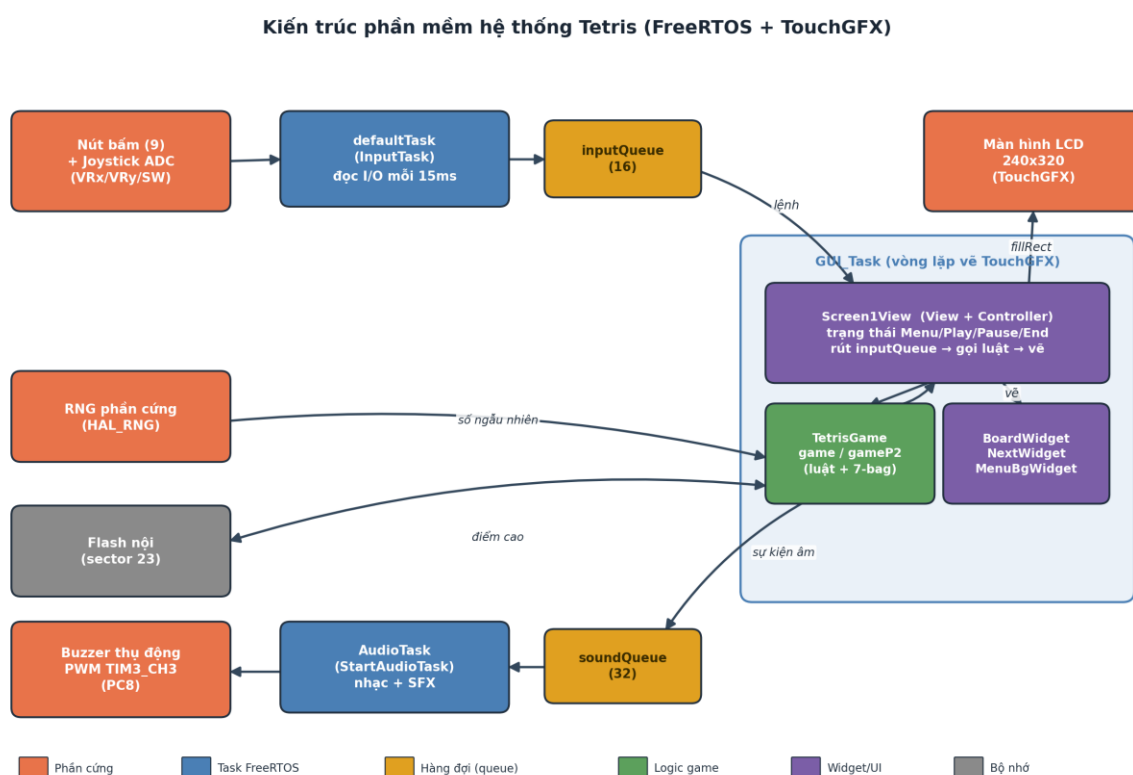
2.3.4. Cấu hình PWM phát âm thanh qua buzzer (Yêu cầu kỹ thuật 2)

Âm thanh được tạo bằng sóng vuông PWM trên kênh TIM3_CH3 (chân PC8), cấu hình trong CubeMX: Timers → TIM3 → Channel 3 → PWM Generation CH3, Prescaler = 89 (đưa xung nhịp timer về 1 MHz với APB1 timer clock 90 MHz). Auto-Reload Register (ARR) và giá trị so sánh (Compare) được tính lại theo từng nốt nhạc để đổi tần số phát ra ở buzzer (duty cycle luôn giữ 50%), đáp ứng yêu cầu kỹ thuật bắt buộc số 2 của đề tài.

CHƯƠNG 3: XÂY DỰNG CHƯƠNG TRÌNH VÀ TRIỂN KHAI CÀI ĐẶT

3.1. Sơ đồ hệ thống

Với những yêu cầu đã đặt ra khi lên ý tưởng về đề tài, nhóm chúng em đã xây dựng sơ đồ toàn cảnh hệ thống như sau:



Hình 8: Sơ đồ kiến trúc phần mềm hệ thống

Kiến trúc phần mềm gồm các thành phần chính sau:

1. Screen1View: lớp View kiêm luôn vai trò điều khiển (Controller) — giữ trạng thái trò chơi (Menu/Play/Pause/End), rút lệnh từ inputQueue, gọi luật chơi và vẽ lại giao diện mỗi tick.
2. TetrisGame: lớp lõi cài đặt luật Tetris (va chạm, xoay, khoá khối, xoá hàng, tính điểm, level, 7-bag RNG, hold, ghost); Screen1View sở hữu 1 instance (hoặc 2 instance song song ở chế độ 2 người – 2P Versus).

3. BoardWidget, NextWidget, MenuBgWidget: các custom widget TouchGFX tự vẽ bàn chơi, khối kẻ tiếp/khối giữ và nền menu bằng fillRect() (không dùng ảnh bitmap dựng sẵn).
4. defaultTask (InputTask) và GUI_Task: 2 task FreeRTOS — defaultTask đọc 8 nút bấm chính + joystick mỗi ~15 ms; GUI_Task chạy vòng lặp vẽ của TouchGFX, bên trong gọi Screen1View::handleTickEvent() để xử lý lệnh và cập nhật game mỗi khung hình.
5. inputQueueHandle và soundQueueHandle: 2 hàng đợi FreeRTOS (osMessageQueue) trung chuyển lệnh điều khiển (16 phần tử) và sự kiện âm thanh (32 phần tử) giữa các task, tách rời phần đọc phần cứng khỏi phần xử lý luật chơi và âm thanh.

3.2. Luồng hệ thống

Hệ thống bao gồm 3 luồng hoạt động chính:

1. Nhận dữ liệu từ tác nhân người chơi
2. Xử lý logic của game và xây dựng giao diện
3. Phát âm thanh hiệu ứng và nhạc nền

3.2.1. Nhận dữ liệu từ tác nhân người chơi

8 nút bấm chính (LEFT=PG2, RIGHT=PG3, ROT_CW=PC11, ROT_CCW=PD4, SOFT=PD2, HARD=PD7, HOLD=PD5, JOY_SW=PG9) và 2 kênh ADC1 của joystick (VRx=PC3, VRy=PA7, đọc liên tục qua DMA) được đọc trong task defaultTask (hàm StartDefaultTask, đóng vai trò InputTask) mỗi 15 ms. Sau khi debounce và xử lý DAS/ARR (giữ nút ~150 ms rồi tự lặp mỗi ~45 ms để di chuyển mượt), lệnh rời rạc (Trái/Phải/Xoay/Roi nhanh...) được đẩy vào hàng đợi inputQueueHandle bằng osMessageQueuePut().

```
/* Core/Src/main.c – StartDefaultTask(): đọc nút bấm (pull-up nội, nhấn = mức 0)
*/
uint8_t bL = (HAL_GPIO_ReadPin(BTN_LEFT_GPIO_Port, BTN_LEFT_Pin) ==
GPIO_PIN_RESET);
```

```
uint8_t bR = (HAL_GPIO_ReadPin(BTN_RIGHT_GPIO_Port, BTN_RIGHT_Pin) ==
GPIO_PIN_RESET);

uint8_t bCW = (HAL_GPIO_ReadPin(BTN_ROT_CW_GPIO_Port, BTN_ROT_CW_Pin) ==
GPIO_PIN_RESET);

// ... tương tự cho bCCW, bSoft, bHard, bSW (joystick SW), bCCW2 (P2)

/* Joystick (0..4095, giữa ~2048) – ngưỡng + vùng chết */
uint16_t vx = g_adcJoy[0], vy = g_adcJoy[1]; /* đọc qua ADC1 2 kênh, DMA circular
*/
uint8_t jL = (vx < 1200), jR = (vx > 2900);
uint8_t jDown = (vy > 2900), jUp = (vy < 1200);

/* debounce + DAS/ARR rồi đẩy lệnh vào hàng đợi (rút gọn) */
if (emit1 && inputQueueHandle != NULL) {
    uint8_t code = (uint8_t)a; /* 0..5 khớp enum tetris::Command
*/
    osMessageQueuePut(inputQueueHandle, &code, 0, 0);
}
osDelay(15);
```

(Trích StartDefaultTask() — Core/Src/main.c: đọc 8 nút chính + joystick, đẩy lệnh vào inputQueueHandle)

Bên cạnh việc đọc input, hàm MX_FREERTOS_Init() cũng khởi tạo 2 hàng đợi và 3 task FreeRTOS nêu ở mục 3.1:

```
/* Core/Src/main.c – MX_FREERTOS_Init(): tạo 2 hàng đợi + 3 task */
inputQueueHandle = osMessageQueueNew(16, sizeof(uint8_t), NULL);
soundQueueHandle = osMessageQueueNew(32, sizeof(uint8_t), NULL);

defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
/* InputTask */
GUI_TaskHandle = osThreadNew(TouchGFX_Task, NULL, &GUI_Task_attributes);
/* vẽ + game logic */
```

```
audioTaskHandle = osThreadNew(StartAudioTask, NULL, &audioTask_attributes);  
/* âm thanh */
```

(Trích MX_FREERTOS_Init() — Core/Src/main.c: tạo inputQueue/soundQueue và 3 task)

Lệnh trong inputQueueHandle được Screen1View rút ra và xử lý mỗi khung hình (tick) của TouchGFX, trong hàm handleTickEvent(). Hàm này dùng osMessageQueueGet() ở chế độ non-blocking (timeout = 0) để không làm chậm vòng lặp vẽ, rồi phân phối lệnh theo trạng thái màn hình hiện tại (Menu / đang chơi / kết thúc):

```
/* TouchGFX/gui/src/screen1_screen/Screen1View.cpp */  
void Screen1View::handleTickEvent()  
{  
    uint8_t code;  
    while (inputQueueHandle != NULL &&  
           osMessageQueueGet(inputQueueHandle, &code, NULL, 0) == osOK)  
    {  
        if (code == tetris::CMD_PAUSE_CODE) { togglePause(); }  
        else if (state == VS_PLAY && !paused && code <= tetris::CMD_HARD_DROP)  
        {  
            game.command((tetris::Command)code); /* áp lệnh vào luật chơi */  
            changed = true;  
        }  
        // ... các nhánh còn lại xử lý Menu / End / chế độ 2 người (PvP)  
    }  
    // ... onGravityTick(), cập nhật HUD, invalidate BoardWidget  
}
```

(Trích Screen1View::handleTickEvent() — rút inputQueue và áp lệnh vào TetrisGame)

Khi đang chơi (VS_PLAY), lệnh được chuyển thành lời gọi game.command(...) trên đối tượng TetrisGame tương ứng (game cho người chơi 1, gameP2 cho người chơi 2 ở chế độ 2P Versus), làm khối di chuyển/xoay/roi nhanh theo đúng luật Tetris.

3.2.2. Xử lý logic của game và xây dựng giao diện

Khác với sơ đồ dự kiến ban đầu có một gameTask riêng, trong bản cài đặt thực tế phần xử lý luật chơi được gộp trực tiếp vào GUI_Task: mỗi khung hình, Screen1View::handleTickEvent() vừa rút lệnh từ inputQueue, vừa gọi trọng lực (onGravityTick()) và cập nhật đối tượng TetrisGame, rồi mới vẽ lại BoardWidget — nhờ vậy trạng thái game và khung hình luôn đồng bộ, không cần cơ chế đồng bộ hoá thêm giữa 2 task.

TetrisGame áp luật va chạm, khoá khối khi chạm đáy, quét và xoá hàng đầy, cộng điểm theo bảng $\{0, 100, 300, 500, 800\} \times \text{level}$, tăng level mỗi 10 hàng xoá được (chi tiết luật chơi ở mục 3.3). Khối được sinh ra theo thuật toán 7-bag dùng RNG phần cứng — đúng yêu cầu kỹ thuật bắt buộc số 1 của đề tài:

```
// TouchGFX/gui/src/game/TetrisGame.cpp

// Thuật toán 7-bag: mỗi túi chứa đúng 1 lần đủ 7 khối, xáo trộn -> công bằng,
// không bị "đói" khối, không ra quá nhiều khối trùng liên tiếp.
void TetrisGame::refillBag()
{
    for (int i = 0; i < NUM_PIECES; i++)
        bag[i] = (uint8_t)i;

    // Xáo trộn Fisher-Yates bằng RNG phần cứng
    for (int i = NUM_PIECES - 1; i > 0; i--)
    {
        uint32_t j = randomValue() % (uint32_t)(i + 1);    // randomValue() ->
hw_rand() trên board
        uint8_t t = bag[i];
        bag[i] = bag[j];
        bag[j] = (uint8_t)t;
    }
    bagIdx = 0;
}
```

(Trích TetrisGame::refillBag() — thuật toán 7-bag dùng RNG phân cứng)

3.3. Luật chơi và thuật toán

3.3.1. Bàn chơi và khối

Bàn chơi có kích thước 10×20 ô (COLS×ROWS); ở chế độ 1 người mỗi ô vuông 16 px nên bàn 160×320 phủ kín chiều cao màn 240×320, chừa cột bên phải rộng 80 px cho HUD (chế độ 2 người dùng ô 12 px cho 2 bàn song song). Game có đủ 7 loại khối tetromino chuẩn (I, O, T, S, Z, J, L), mỗi khối lưu dưới dạng ma trận hình spawn 4×4; các góc xoay được tính bằng ánh xạ tọa độ ngược (hàm cellFilled()) thay vì lưu sẵn 4 trạng thái xoay cho từng khối. Khối O không xoay (đối xứng vuông nên không cần thiết); các khối còn lại xoay cơ bản — phép xoay bị chặn nếu vị trí mới va chạm tường/khối khác. Nhóm chưa cài đặt bảng wall-kick chuẩn SRS (Super Rotation System) cho phép xoay sát tường/đáy như game Tetris thương mại; đây là điểm nêu ở mục Hướng phát triển (4.3).

3.3.2. Va chạm, khoá khối và xoá hàng

Hàm collides() kiểm tra 4 ô của khối tại vị trí/góc xoay đang xét: nếu ra ngoài tường/đáy hoặc trùng ô đã có khối trong board[][] thì coi là va chạm. Khi khối không thể hạ thêm (va chạm ngay phía dưới), lockPiece() ghi khối vào board[][] và phát âm hiệu ứng “khóa khối” — bản cài đặt hiện tại khoá ngay lập tức, chưa có khoảng trễ (lock delay ~0,5 s) như dự kiến ban đầu. Sau khi khoá, clearLines() quét bàn từ hàng dưới lên; hàng nào đầy thì dồn toàn bộ các hàng phía trên xuống 1 bậc và kiểm tra lại chính hàng đó (vì vừa có hàng mới dồn xuống).

3.3.3. Tính điểm, cấp độ và trọng lực

Điểm được cộng theo số hàng xoá cùng lúc, nhân với level hiện tại — đúng bảng chuẩn Guideline {0, 100, 300, 500, 800} cho {0,1,2,3,4 hàng}; ngoài ra soft drop được +1 điểm/ô, hard drop được +2 điểm/ô theo quãng đường rơi. Cứ xoá đủ 10 hàng thì level tăng 1 (level = 1 + lines/10), khiến trọng lực nhanh dần: số khung hình cần để khối rơi 1 ô là $\max(4, 48 - 4 \times (\text{level} - 1))$ ở 60 khung hình/giây — khoảng 0,8 s/ô ở level 1, giảm

dần tới mức sàn $\sim 0,07$ s/ô. Riêng chế độ Zen giữ cố định 40 khung hình/ô bất kể level, đúng tinh thần “chơi thư giãn, không tăng độ khó” đã đề ra.

3.3.4. Hold và Ghost piece

Người chơi có thể giữ (hold) khối hiện tại lại để dùng sau, đổi chỗ với khối đang giữ (nếu có); mỗi khối mới chỉ được giữ 1 lần (cờ holdUsed). Ghost piece — bóng chỉ vị trí khối sẽ rơi tới nếu thả thẳng — được tính bằng cách chiếu khối hiện tại thẳng xuống tới khi va chạm, rồi renderTo() ghi các ô bóng vào lưới hiển thị với mã màu +8 so với khối thật; BoardWidget vẽ các ô này ở độ sáng thấp hơn ($\sim 40\%$) để phân biệt với khối đang điều khiển.

3.4. Chế độ chơi

Đề tài yêu cầu độ khó tăng dần chứ không cố định. Nhóm cài đặt đủ 4 chế độ theo kế hoạch, cùng 1 chế độ bổ sung ngoài dự kiến ban đầu — 2P Versus (2 người chơi cùng lúc trên 2 bàn song song, dùng nút bấm và joystick tách biệt cho từng người):

Chế độ	Luật chơi	Điều kiện kết thúc
Marathon	Chơi cổ điển, level tăng dần theo số hàng đã xóa, trọng lực nhanh dần	Khối mới sinh ra bị chồng lên khối cũ (top out)
Sprint (40 lines)	Đua xóa đủ 40 hàng nhanh nhất có thể	Đạt 40 hàng — ghi nhận thời gian hoàn thành
Ultra (2:00)	Ghi điểm tối đa trong 2 phút, level vẫn tăng theo số hàng	Hết 2 phút (120 giây = 7200 khung hình ở 60 fps)
Zen	Trọng lực nhẹ, cố định (không tăng theo level); không bao giờ thua — bàn tự dọn sạch khi đầy	Người chơi tự thoát về Menu

2P Versus (bổ sung)	Hai người chơi độc lập trên 2 bàn song song — nút bấm điều khiển người chơi 1, joystick điều khiển người chơi 2	Ai top-out trước thua; nếu cùng lúc, ai nhiều hàng hơn thắng
---------------------	---	--

3.5. Lưu điểm cao (High score)

Kỷ lục của cả 4 chế độ chơi 1 người được lưu bền vào 1 sector Flash nội của STM32F429 (sector 23, bank 2, địa chỉ 0x081E0000) — chọn bank 2 để việc xoá/ghi Flash không làm treo việc CPU đọc lệnh hiển thị LCD đang chạy ở bank 1 (read-while-write). Dữ liệu được đọc lại mỗi khi khởi động và chỉ ghi lại khi có kỷ lục mới (xoá cả sector rồi ghi lại từ đầu), kèm 1 giá trị “magic number” để nhận biết Flash đã từng được ghi hay chưa:

```
/* Core/Src/main.c */

#define HISCORE_ADDR    0x081E0000UL

#define HISCORE_MAGIC   0x54455452UL    /* 'TETR' - nhận biết dữ liệu hợp lệ */

/* Đọc 4 điểm cao (4 chế độ) từ Flash. Lần đầu (chưa có magic) -> trả 0. */
void hiscore_load(uint32_t* out4)
{
    const volatile uint32_t* p = (const volatile uint32_t*)HISCORE_ADDR;

    if (p[0] == HISCORE_MAGIC) { for (int i = 0; i < 4; i++) out4[i] = p[1 + i]; }
    else                          { for (int i = 0; i < 4; i++) out4[i] = 0; }
}

/* Xoá sector 23 rồi ghi magic + 4 điểm cao. Chỉ gọi khi có kỷ lục mới (hiếm). */
void hiscore_save(const uint32_t* in4)
{
    FLASH_EraseInitTypeDef er; uint32_t secErr = 0;

    HAL_FLASH_Unlock();
}
```

```
er.TypeErase = FLASH_TYPEERASE_SECTORS; er.Sector = FLASH_SECTOR_23;

er.NbSectors = 1; er.VoltageRange = FLASH_VOLTAGE_RANGE_3;

if (HAL_FLASHEx_Erase(&er, &secErr) == HAL_OK) {

    HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, HISCORE_ADDR, HISCORE_MAGIC);

    for (int i = 0; i < 4; i++)

        HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD, HISCORE_ADDR + 4u + (uint32_t)i *
4u, in4[i]);

}

HAL_FLASH_Lock();

}
```

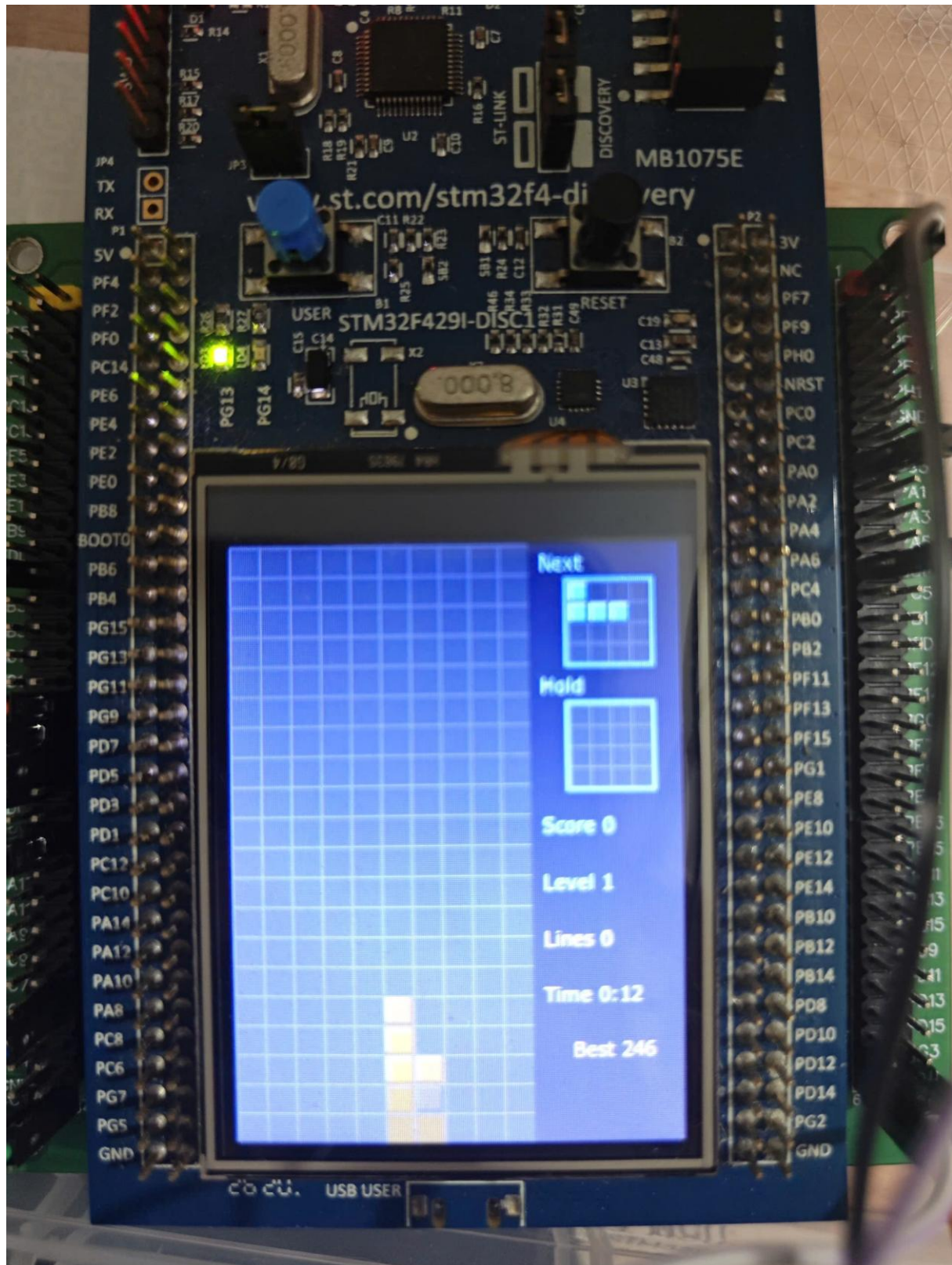
(Trích hiscore_load()/hiscore_save() — Core/Src/main.c: đọc/ghi điểm cao vào Flash nội)

CHƯƠNG 4: ĐÁNH GIÁ TỔNG QUAN

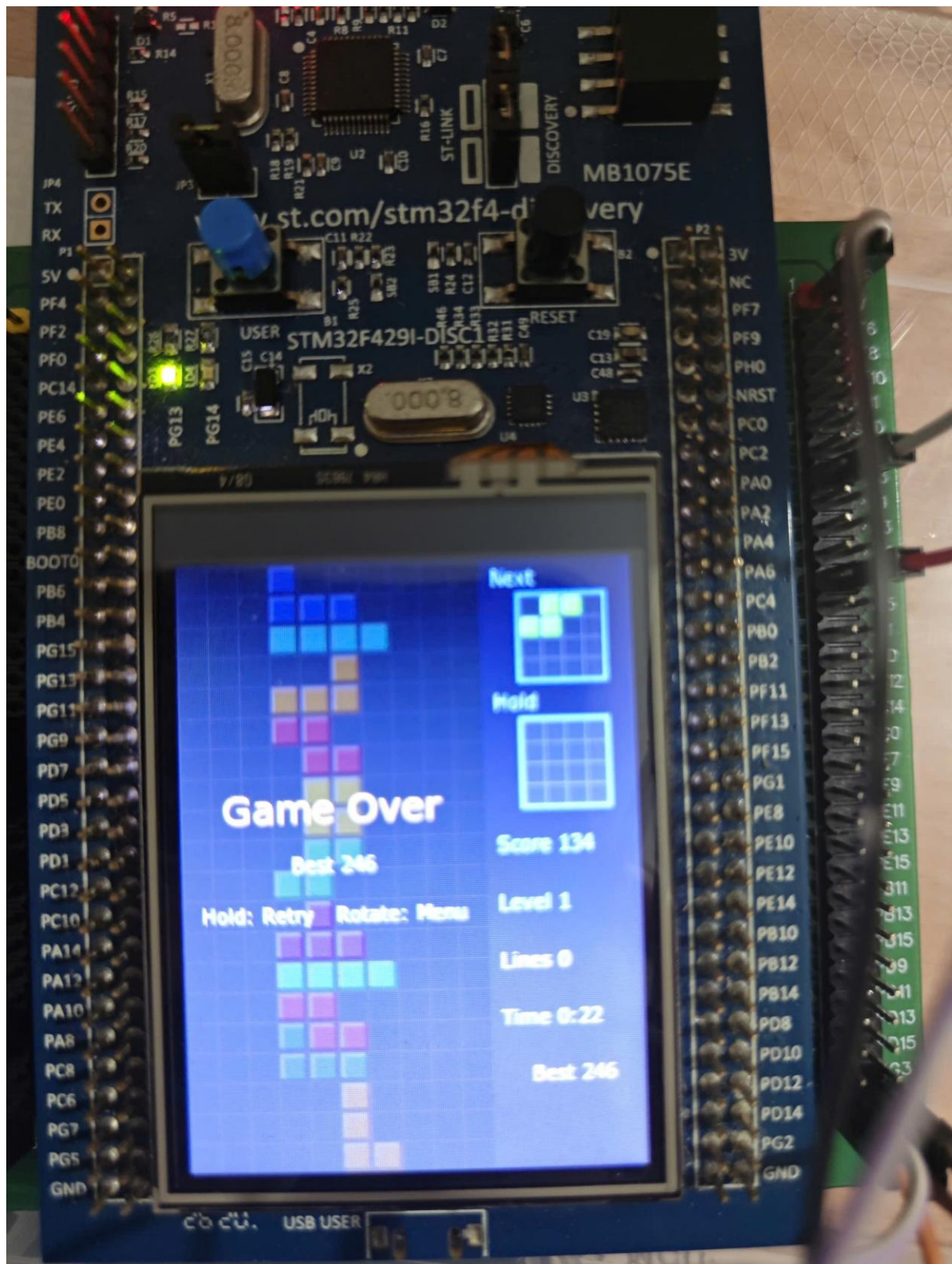
4.1. 4.1. Sản phẩm demo



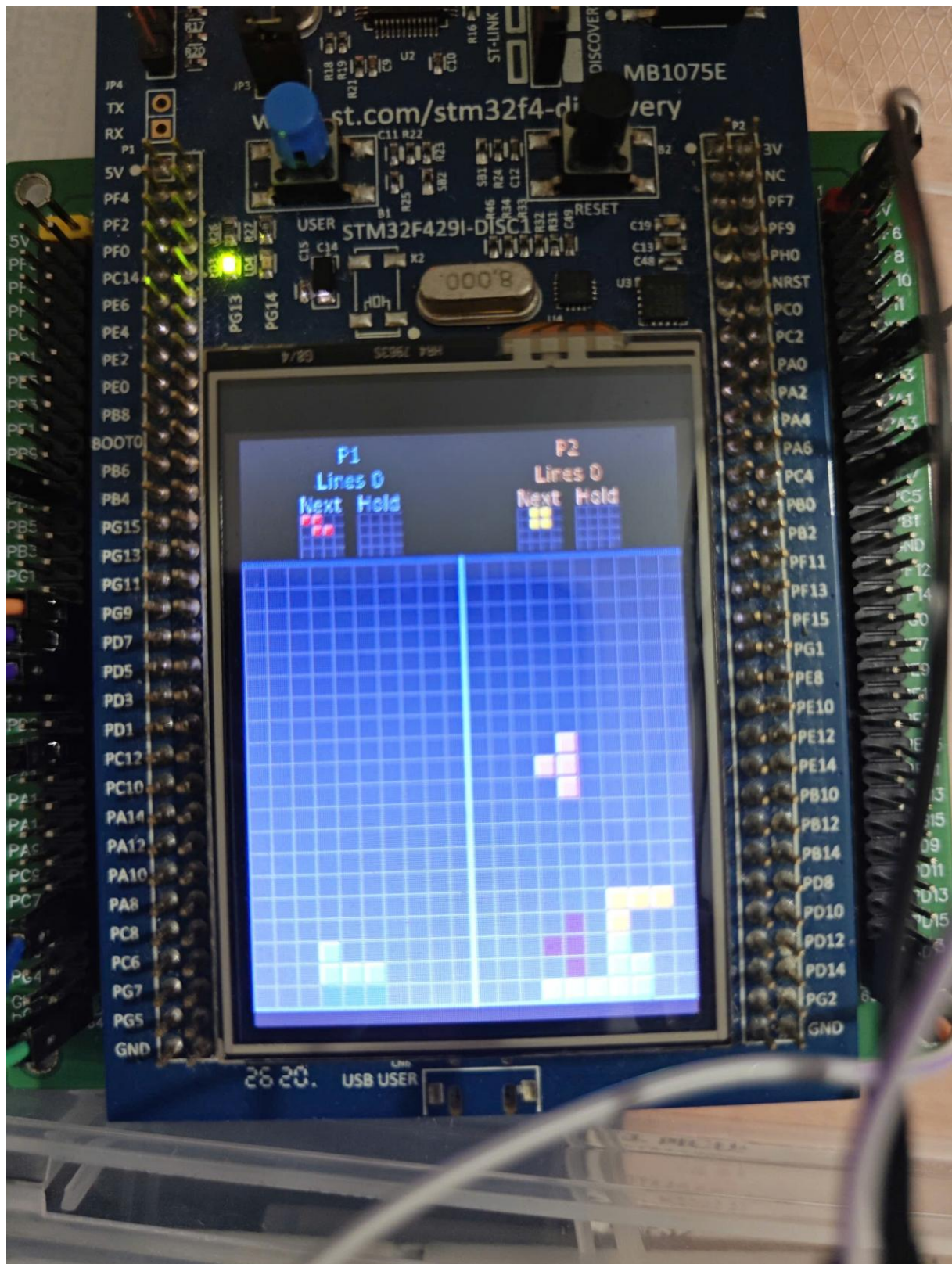
Hình 9: Giao diện Menu (giao diện mở đầu, chọn chế độ chơi)



Hình 10: Giao diện khi chơi (HUD)



Hình 11: Giao diện Game Over / kết thúc lượt chơi



Hình 12 (bổ sung): Giao diện chế độ 2 người (2P Versus)

4.2. Đánh giá hệ thống

Trong khuôn khổ đồ án môn học, nhóm chúng em đã thiết kế được mô hình đạt được các yêu cầu đề ra:

- Luật chơi & tính đúng đắn: 7 khối tetromino rơi, xoay, va chạm, xoá hàng, tính điểm/level hoạt động chính xác và ổn định qua nhiều ván chơi liên tục.
- Tính ngẫu nhiên công bằng: khối được sinh bằng thuật toán 7-bag dùng RNG phần cứng, không lặp/đổi một loại khối nào quá lâu — đáp ứng đúng yêu cầu kỹ thuật bắt buộc số 1.
- Âm thanh: hiệu ứng (di chuyển, xoay, khoá khối, xoá hàng, Tetris, lên level, game over) và nhạc nền Korobeiniki phát đúng thời điểm qua buzzer PWM, không bị trễ hay giật — đáp ứng yêu cầu kỹ thuật bắt buộc số 2.
- Điều khiển kép: 8 nút bấm chính và joystick hoạt động đồng thời, gộp chung vào 1 hàng đợi lệnh, độ trễ thấp (debounce ~15 ms, DAS/ARR mượt); ở chế độ 2P Versus, 2 nguồn input tách bạch rõ ràng cho 2 người chơi.
- Lưu điểm cao bền vững: kỷ lục của cả 4 chế độ chơi 1 người được ghi vào 1 sector Flash nội, giữ nguyên sau khi rút nguồn.

Các vấn đề còn thiếu sót:

- Xoay khối chưa có wall-kick (SRS): khối chỉ xoay được khi vị trí mới không va chạm, chưa thử các phép dịch chuyển sát tường/đáy theo chuẩn Super Rotation System, nên đôi lúc không xoay được sát tường như game Tetris thương mại.
- Chưa có lock delay: khối khoá ngay khi vừa chạm đáy/khối khác, không có khoảng trễ ~0,5 s để người chơi kịp trượt/xoay chỉnh trước khi khoá.
- Next chỉ xem trước được 1 khối (chưa đủ 3 khối như dự kiến ban đầu), và màn hình nhỏ (240×320) giới hạn không gian hiển thị HUD.

4.3. Hướng phát triển

- Bổ sung wall-kick chuẩn SRS và lock delay để xoay/đặt khối linh hoạt hơn.
- Tăng số khối xem trước (Next) lên 3, thêm combo/back-to-back Tetris, tính điểm T-spin.

- Thêm hiệu ứng hình ảnh khi xoá hàng, rung màn hình khi hard drop, đèn LED PG13/PG14 nhấp nháy khi đạt Tetris/lên level.

PHỤ LỤC

1. Phân công nhiệm vụ

Nhiệm vụ	Người thực hiện
Tìm hiểu đề tài, lên kế hoạch (docs/KE_HOACH_Tetris.md), lên sơ đồ chân (docs/P0.2_PINMAP_CubeMX.md)	Cả nhóm
Khởi tạo project CubeMX/CubeIDE, tích hợp khung TouchGFX	Đỗ Văn Việt
Cấu hình & lập trình ngoại vi phần cứng trong main.c (RNG, ADC joystick, PWM buzzer TIM3, GPIO 10 nút, Flash lưu điểm cao)	Đỗ Văn Việt
Xây dựng lõi luật chơi Tetris (TetrisGame: va chạm, xoay, khoá khối, xoá hàng, tính điểm, level, 7-bag, hold, ghost piece)	Nguyễn Tuấn Cảnh
Thiết kế giao diện TouchGFX (BoardWidget, NextWidget, MenuBgWidget, màn Menu/HUD/Pause/Game Over, chế độ 2P Versus)	Nguyễn Tuấn Cảnh
Tích hợp cầu nối âm thanh (audioTask, nhạc nền Korobeiniki, sự kiện SFX)	Đỗ Văn Việt
Kiểm thử, cân bằng thông số (DAS/ARR, tốc độ rơi theo level), quay video demo	Cả nhóm

Viết báo cáo, tổng hợp tài liệu	Cả nhóm
---------------------------------	---------

2. Tài liệu tham khảo

- Bài giảng học phần “Hệ nhúng” – IT4210 của TS. Ngô Lam Trung.
- <https://gociter.wordpress.com/2018/12/09/he-thong-nhung-giao-tiep-chuot-va-ban-phim-su-dung-kit-stm32f429-discovery/>
- <https://hocarm.org/huong-dan-stm32cubeide/>
- Youtube Lập trình STM32 trên STM32CubeIDE:
https://www.youtube.com/playlist?list=PLyD_mbw_VznOxKMBg5CjJ6OWvDzPNR8F4
- Schematic Discovery kit with STM32F429ZI MCU:
<https://docs.rs-online.com/b72f/0900766b815247be.pdf>

3. Mã nguồn (source code) và các resource của đồ án

- Mã nguồn chương trình:
<https://github.com/dovanviet04112004/tetrisgame.git>
- Tài liệu kế hoạch & sơ đồ chân nội bộ: docs/KE_HOACH_Tetris.md, docs/P0.2_PINMAP_CubeMX.md (trong repo mã nguồn)
- Link video demo: [ĐIỀN LINK VIDEO DEMO SAU KHI QUAY]
- IDE lập trình: STM32CubeIDE
- Công cụ vẽ sơ đồ, biểu đồ: <https://app.diagrams.net>
- Bản thiết kế mạch điện KIT STM32F429I-DISC1:
<https://dtxvn.com/san-pham/stm32f429i-disc1/>
<https://www.waveshare.com/32f429idiscovery.htm>